Cost-Sensitive SVM for Credit Card Fraud Detection: From Costs to Algorithms

Group Project Research Proposal

Group 4 · Topic: Convex Optimization · Date: 22 October 2026

Presentation: Week 9, 27 October 2026 (per the latest assignment schedule; to be confirmed with the teaching team)

PROPOSAL AT A GLANCE

Real problem: credit card fraud detection with extreme class imbalance and asymmetric costs

Methods: SMO · Pegasos · fixed-ρ ADMM · PA-ADMM · interior-point

Metric: time-to-cost-tolerance · PR-AUC · recall at fixed FPR · cost per example

Model: cost-sensitive soft-margin SVM — a convex QP with class-dependent penalties

Data: ULB credit card fraud dataset (284,807 transactions) + synthetic stress tests

Deliverables: report · 40-minute presentation · reproducible code

Abstract

Credit card fraud detection is a binary screening problem with extreme class imbalance (0.172% fraud in the ULB dataset), asymmetric error costs, noisy labels, and a training time budget. We model the task as a **cost-sensitive soft-margin SVM**, a convex quadratic program whose class-dependent penalties are derived from a false-negative/false-positive cost matrix. We derive the primal, dual, KKT conditions, and an ADMM splitting, then implement SMO, Pegasos, fixed-ρ ADMM, a warm-started adaptive ADMM (PA-ADMM), and an interior-point baseline under one interface. The evaluation uses the 284,807-transaction ULB dataset and controlled synthetic stress tests for fraud rate, label noise, and cost ratio. The primary metric is **time-to-cost-tolerance**: wall-clock time to reach a target weighted objective gap or cost per example. We also report PR-AUC, recall at false-positive rates of 0.1% and 1%, and sensitivity to the cost ratio. The expected outcome is a cost-aware regime map: cost-sensitive weighting should reduce total cost under extreme imbalance, and PA-ADMM should reduce time-to-cost-tolerance relative to cold-start ADMM while remaining competitive with Pegasos on large n. Deliverables are a report, a 40-minute presentation, and reproducible code.

1. Introduction

1.1 A real problem: fraud detection under extreme imbalance

Credit card fraud detection is a concrete, high-stakes binary decision. For each transaction, a system must decide whether to flag it as fraudulent. A false negative (a missed fraud) can cost the issuer the transaction amount and chargeback fees; a false positive (a declined legitimate transaction) causes customer friction and may lose future business. The two errors are not symmetric, and the asymmetry is the core of the problem, not an afterthought.

The ULB credit card dataset illustrates the difficulty: 284,807 transactions, of which only 492 (0.172%) are fraudulent. A classifier that predicts “legitimate” for every transaction achieves 99.828% accuracy while catching zero fraud. Accuracy is therefore the wrong metric. The dataset also has noisy labels (some fraud is never reported), anonymized PCA features (V1–V28) plus Time and Amount, and a natural time ordering that matters for deployment.

1.2 From application costs to a convex optimization model

We encode the cost asymmetry as class-dependent penalties in a soft-margin SVM. Let c_{FN} be the cost of a missed fraud and c_{FP} the cost of a false alarm. For a training point with label y_i, set $C_i = C \cdot w_{y_i}$, where $w_+/w_- = c_{\mathrm{FN}}/c_{\mathrm{FP}}$ and the weights are normalized to have mean one. The resulting primal is

$$\min_{w,b,\xi} \ \tfrac{1}{2}\|w\|^2 + \sum_{i=1}^{n} C_i \xi_i \quad \text{s.t.} \quad y_i(w^\top \varphi(x_i) + b) \geq 1 - \xi_i, \ \ \xi_i \geq 0.$$

This is still a convex QP; only the upper bounds in the dual change. The model is interpretable: a higher c_{FN} makes missed fraud more expensive and pushes the classifier toward higher recall. The application-specific assumptions are explicit and can be stress-tested by varying the cost ratio.

1.3 Optimization challenge and research gap

The fraud dataset stresses a specific optimization regime: large n (284,807), moderate d (30), extreme class imbalance, noisy labels, and asymmetric costs. Existing solvers have complementary weaknesses. SMO is accurate and kernel-friendly but requires $O(n^2)$ kernel memory, which is prohibitive at 284k transactions. Pegasos is fast per epoch with $O(d)$ cost, but its stochastic noise near the optimum can be costly when the positive class is rare. ADMM is stable but sensitive to its penalty parameter and slow from a cold start. Interior-point methods are accurate but scale poorly. Most fraud-detection studies report accuracy or AUC without a cost-aware, time-to-tolerance comparison of the underlying optimization algorithms.

A second issue is evaluation. With 0.172% positives, a single train/test split is noisy, and accuracy is misleading. A useful study must use PR-AUC, recall at fixed false-positive rates, cost per example, multiple seeds, and a cost-ratio sweep. It must also separate algorithmic differences from data leakage, threshold choice, and tuning budgets.

1.4 Proposed approach and scope

The project does **not** require a new algorithm. Its primary contribution is an application-grounded cost-sensitive model and a fair, cost-aware comparison of representative solvers. In addition, we evaluate **PA-ADMM** (Pegasos-warm-started adaptive ADMM) as a candidate method: Stage 1 runs a few weighted Pegasos epochs to obtain a cheap rough solution, and Stage 2 initializes ADMM at that point with residual-balanced adaptive ρ and over-relaxation. The hybrid is a design choice to be tested, not an assumption; its components are ablated.

1.5 Research questions

- **RQ1.** Does cost-sensitive weighting reduce total cost relative to an unweighted SVM under extreme imbalance and label noise?
- **RQ2.** How do SMO, Pegasos, fixed-ρ ADMM, PA-ADMM, and an interior-point baseline compare in time-to-cost-tolerance across n, fraud rate, and cost ratio?
- **RQ3.** Does PA-ADMM reduce time-to-cost-tolerance relative to cold-start ADMM, and which component—warm start, adaptive ρ, or over-relaxation—contributes most?
- **RQ4.** Does a lower weighted training objective translate into lower test cost under label noise?
- **RQ5.** What is the cost-aware regime map, including the crossover between dual and primal methods as n grows?

1.6 Objectives

1. Translate the fraud cost matrix into a cost-sensitive SVM QP with class-dependent penalties.
2. Derive the primal, dual, KKT conditions, and the ADMM splitting with per-sample weights.
3. Implement SMO, Pegasos, fixed-ρ ADMM, PA-ADMM, and a CVXPY/ECOS interior-point baseline under one interface.

4. Benchmark on the ULB fraud dataset and controlled synthetic stress tests with cost-aware metrics and statistical tests.
5. Deliver a report, a 40-minute presentation, and a reproducible code repository.

1.7 Scope and assumptions

We study one application: credit card fraud detection. The cost ratios are illustrative assumptions, not a bank's actual loss model; we therefore report a sensitivity analysis over c_{FN}/c_{FP} rather than a single number.

The main experiments use the linear kernel; the RBF kernel is studied on a stratified 10k subset as a qualitative kernel check. SMO uses stratified subsets where its $O(n^2)$ kernel cache would exceed memory; Pegasos and ADMM/PA-ADMM run on the full data. Interior-point is evaluated on a stratified subset. The project is an empirical modeling and comparison study, not a deployment system or a new convergence proof.

2. Problem formulation

2.1 Costs and class weights

Let c_{FN} and c_{FP} be the costs of a missed fraud and a false alarm. The expected cost of a classifier is $c_{FN} \cdot FN + c_{FP} \cdot FP$. To encode this in SVM training, set $C_i = C \cdot w_{y_i}$, where $w_+ = c_{FN}/(c_{FN}+c_{FP}) \cdot 2$ and $w_- = c_{FP}/(c_{FN}+c_{FP}) \cdot 2$ (so the average weight is one). A larger c_{FN} increases the penalty on positive-class slack and pushes the decision toward higher recall.

2.2 Cost-sensitive SVM primal, dual, and KKT

With $D = \{(x_i, y_i)\}_{i=1}^n$, $y_i \in \{-1, +1\}$, the primal is

$$\min_{w,b,\xi} \ \tfrac{1}{2}\|w\|^2 + \Sigma_i C_i \xi_i \quad \text{s.t.} \quad y_i(w^\top \varphi(x_i) + b) \geq 1 - \xi_i, \ \ \xi_i \geq 0.$$

The dual is

$$\max_{\alpha} \ \Sigma_i \alpha_i - \tfrac{1}{2} \Sigma_i \Sigma_j \alpha_i \alpha_j y_i y_j K(x_i, x_j) \quad \text{s.t.} \quad 0 \leq \alpha_i \leq C_i, \ \ \Sigma_i \alpha_i y_i = 0.$$

Only the box constraints change relative to the standard SVM, so the same KKT reasoning applies: $\alpha_i = 0$ for points outside the margin, $0 < \alpha_i < C_i$ for margin support vectors, and $\alpha_i = C_i$ for bounded support vectors. Strong duality holds because the primal is a convex QP satisfying Slater's condition.

2.3 ADMM splitting with class weights

Introduce $z_i = y_i(w^\top x_i + b)$. The primal becomes

$$\min_{w,b,z} \ \tfrac{1}{2}\|w\|^2 + \Sigma_i C_i \max(0, 1 - z_i) \quad \text{s.t.} \quad z_i = y_i(w^\top x_i + b).$$

With dual variable u and penalty $\rho > 0$, the augmented Lagrangian is

$$L_\rho = \tfrac{1}{2}\|w\|^2 + \Sigma_i C_i \max(0, 1 - z_i) + \Sigma_i u_i(z_i - y_i(w^\top x_i + b)) + (\rho/2) \Sigma_i (z_i - y_i(w^\top x_i + b))^2.$$

The (w, b)-update is a ridge regression, solved by a $(d+1) \times (d+1)$ linear system. The z-update is separable: with $a_i = y_i(w^\top x_i + b) - u_i/\rho$, set $z_i = a_i$ if $a_i \geq 1$, otherwise $z_i = \min(1, a_i + C_i/\rho)$. The dual update is $u_i \leftarrow u_i + \rho(z_i - y_i(w^\top x_i + b))$.

2.4 Pegasos with class weights

Pegasos minimizes the weighted primal with stochastic subgradients. At iteration t, sample i, set $\eta_t = 1/(\lambda t)$, and update

$$\text{if } y_i(w^\top x_i + b) < 1\text{:}\quad w \leftarrow (1-\eta_t\lambda)w + \eta_t C_i y_i x_i,\quad b \leftarrow b + \eta_t C_i y_i;\quad \text{else:}\quad w \leftarrow (1-\eta_t\lambda)w.$$

The per-iteration cost is $O(d)$ and memory is $O(d)$; the class weight scales the subgradient, so a few epochs provide a cheap rough solution even when n is large.

2.5 Why convexity and a hybrid are sensible

The cost-sensitive SVM remains convex, so local optima are global and KKT provides a certificate. The algorithmic question is efficiency under the weighted objective. ADMM is stable but its early iterations depend on initialization and ρ; Pegasos is fast but noisy near the optimum. Warm-starting ADMM at a Pegasos iterate combines cheap early progress with stable refinement, while adaptive ρ and over-relaxation reduce tuning and iteration cost. This motivates PA-ADMM, but the comparison against SMO, Pegasos, and interior-point tests whether the hybrid actually wins in time-to-cost-tolerance.

3. Algorithms and proposed method

3.1 Solver families under class weights

Method	Cost-sensitive adaptation	Per-iteration cost	Memory	Expected strength
SMO	box $0 \le \alpha_i \le C_i$; second-order working set	$O(n)$ with kernel cache	$O(n^2)$	small/medium subsets
Pegasos	weighted subgradient $C_i y_i x_i$	$O(d)$	$O(d)$	full 284k, fast epochs
ADMM	z-update uses C_i/ρ	$O(d^3 + nd^2)$	$O(d^2)$	stable, $d = 30$
PA-ADMM	ADMM + Pegasos warm start + adaptive ρ	same as ADMM	$O(d^2)$	candidate hybrid
Interior-point	CVXPY with per-sample weights	$O(n^3)$ worst case	$O(n^2)$	small stratified subset

3.2 PA-ADMM

PA-ADMM runs K epochs of weighted Pegasos to obtain (w_0, b_0), initializes $z_i^0 = y_i(w_0^\top x_i + b_0)$, $u^0 = 0$, $\rho = \rho_0$, and runs ADMM with residual-balanced ρ updates and over-relaxation until the residuals and relative weighted objective gap fall below τ.

Algorithm 1: PA-ADMM for cost-sensitive SVM

1. **Input:** X, y, C_i, Pegasos epochs K, initial penalty ρ_0, over-relaxation α, tolerance τ.
2. Run weighted Pegasos for K epochs $\rightarrow (w_0, b_0)$.
3. Initialize $z_i = y_i(w_0^\top x_i + b_0)$, $u = 0$, $\rho = \rho_0$.
4. **repeat**
5. $(w,b) \leftarrow \operatorname{argmin} \tfrac{1}{2}\|w\|^2 + (\rho/2)\|z - y\odot(Xw + b1) + u/\rho\|^2$.
6. $a \leftarrow y\odot(Xw + b1) - u/\rho$; $z \leftarrow \min(1, a + C_i/\rho)$, with $z_i = a_i$ if $a_i \ge 1$.
7. $u \leftarrow u + \rho(\alpha\, z^{\text{new}} + (1-\alpha)\, z^{\text{old}} - y\odot(Xw + b1))$.
8. If $\|r\| > 10\|s\|$, $\rho \leftarrow 2\rho$; if $\|s\| > 10\|r\|$, $\rho \leftarrow \rho/2$; rescale u.
9. **until** primal/dual residuals and relative weighted objective gap $< \tau$.
10. **Output:** w, b.

Components. The warm start costs $O(Knd)$ and removes part of ADMM's cold-start transient. Adaptive ρ multiplies or divides ρ by 2 when the primal and dual residuals differ by an order of magnitude, and rescales u. Over-relaxation uses $\alpha \in [1.5, 1.8]$ in the dual update. Both are standard acceleration heuristics (Boyd et

al., 2011); their effect is measured, not assumed. The (w, b)-update solves a 31×31 system; with a cached $X^\top X$ and Cholesky factorization, the per-iteration cost is dominated by $O(nd^2) \approx 2.6 \times 10^8$ operations at full n, which is practical.

3.3 Baselines and ablations

We compare five methods under one interface: **SMO**, **Pegasos**, **fixed-ρ ADMM**, **PA-ADMM**, and **interior-point** (CVXPY/ECOS, small/medium data). Ablations isolate (i) warm start $K \in \{0, 5, 10, 20\}$, (ii) adaptive ρ on/off, (iii) over-relaxation $\alpha \in \{1.0, 1.5, 1.8\}$, (iv) initial ρ_0, and (v) cost-sensitive weights versus unweighted training. The last ablation is essential: it tests whether the application modeling actually lowers total cost rather than merely changing the objective.

3.4 Experimental design

Dataset	Source	Transactions	*d*	Fraud rate	Role
ULB credit card	Kaggle / ULB ML Group	284,807	30	0.172%	primary real application
Synthetic fraud	generated	10k–200k	20	0.1–10%	controlled stress tests

We use one application dataset and controlled synthetic stress tests, rather than mixing unrelated domains. The ULB data contain 284,807 transactions and 492 frauds; Time and Amount are standardized on the training fold, V1–V28 are already PCA components. Exact duplicate rows are removed before splitting. We use stratified 70/15/15 splits with five seeds; a chronological 70/30 split is reported as a robustness check because the data have a natural time order.

Cost ratios. The main runs use $c_{FN}/c_{FP} \in \{10, 20, 50\}$; the full sweep includes $\{1, 3, 5, 10, 20, 50\}$. These are illustrative, not a bank's loss model. Class weights are normalized to mean one, and the base penalty $C \in \{0.01, 0.1, 1, 10, 100\}$ is selected on validation cost. We never tune on the test set.

Metrics. The primary metric is **time-to-cost-tolerance**: wall-clock time to reach a relative weighted objective gap $\le 10^{-4}$ (10^{-3} for full data) or a target cost per example. We also report PR-AUC (average precision), recall at false-positive rates of 0.1% and 1%, cost per example, balanced accuracy, primal/dual residuals, iterations, and peak memory. We plot weighted objective versus time and test cost versus training objective.

Hard regimes. We vary the fraud rate by subsampling positives (0.1%, 0.5%, 1%, 5%, 10%); flip 5% and 10% of training labels; sweep the cost ratio; and use a chronological split to expose distribution drift. Timeouts and out-of-memory runs are recorded as failures. Results include bootstrap 95% confidence intervals and Wilcoxon signed-rank tests for paired differences across seeds.

Fairness controls. The same splits, cost ratio, base C, stopping tolerance, single-threaded execution, and hardware are used for all methods. Hyperparameters are selected on validation cost only. We report both per-iteration and per-time behavior, because an algorithm with cheap iterations may win on time while losing on iterations. Wall-clock results are hardware-dependent and will be interpreted with that caveat.

3.5 Evaluation criteria

The comparison is judged on correctness (agreement with LIBSVM/CVXPY within tolerance), cost efficiency (time-to-cost-tolerance and total cost), scalability (runtime and memory versus n), robustness (seed, ρ_0, label noise, fraud rate, cost ratio), and reproducibility (one command reruns all tables and figures).

4. Expected outcomes and contributions

The project will test five hypotheses:

- **H1.** Cost-sensitive weighting reduces total cost relative to an unweighted SVM under extreme imbalance and label noise, with the largest gains at low fraud rates and high cost ratios.

- **H2.** PA-ADMM reduces time-to-cost-tolerance relative to cold-start fixed-ρ ADMM by a significant factor, especially on larger n.
- **H3.** Adaptive ρ reduces sensitivity to ρ_0, over-relaxation reduces the iteration count, and the warm start contributes most when the cold-start transient is large.
- **H4.** Pegasos is fastest per epoch and scales to the full 284k transactions; SMO is strong on stratified subsets; PA-ADMM is competitive with Pegasos at tolerance on the full data.
- **H5.** There is no universal winner: the ranking is regime- and cost-dependent, and a lower weighted training objective does not always imply a lower test cost under label noise.

These are hypotheses to be confirmed or refuted, not claimed results. The expected contributions are: an application-grounded cost-sensitive model with an explicit cost-matrix interpretation; a unified time-to-cost-tolerance benchmark for extreme imbalance; a cost-aware regime map; an ablation of a warm-started hybrid; and an open-source implementation. The deliverables are the project report, the 40-minute presentation, and the code repository.

5. Timeline

The timeline assumes the presentation on Week 9, 27 October 2026, as stated in the latest assignment schedule. If the teaching team confirms a different date, the milestones scale proportionally and the scope remains unchanged.

Date	Milestone
Thu 22 Oct	Scope freeze; repository setup; download/cache ULB data; define cost ratios and class weights; theory outline; Pegasos and ADMM scaffolds
Fri 23 Oct	Primal/dual draft and CVXPY baseline; SMO v1; weighted Pegasos v1; fixed-ρ ADMM v1; PA-ADMM v1
Sat 24 Oct	SMO tuned and checked against LIBSVM on subsets; PA-ADMM tuning and ablations; first full-data matrix; report draft
Sun 25 Oct	Cost-ratio sweep, fraud-rate and label-noise stress tests; chronological split; plots; report sections (Quiz 2 is also due)
Mon 26 Oct	Code freeze; final runs; slides assembled; timed 40-minute rehearsal
Tue 27 Oct	Final rehearsal; presentation

6. Division of labor

The project is divided into four equal work packages (25% each). Each member owns one work package, contributes to code review, and presents for 10 of the 40 minutes. The split balances application modeling and theory, algorithm implementation, and experimental work.

WP #	Owner	Responsibilities	Deliverables	Report sections	Talk	Effort
1	Member 1	Fraud cost model and application story; primal/dual/KKT; weighted Pegasos; report integration	Cost-matrix note; theory draft; Pegasos module; merged report	Introduction; Application; Formulation; Theory; Pegasos	10 min	**25%**
2	Member 2	SMO with per-class bounds; kernel subset; stopping rule; sanity checks	SMO module; convergence plots; unit tests	SMO	10 min	**25%**
3	Member 3	ADMM and PA-ADMM; adaptive ρ; over-relaxation; ablations	Two modules; ablation study; runtime plots	ADMM; PA-ADMM	10 min	**25%**
4	Member 4	Data pipeline; cost-ratio and stress-test protocol; evaluation; visualization; slides	Evaluation harness; results CSV; figures; slides	Experiments; Results; Discussion; Conclusion	10 min	**25%**

Shared responsibilities. All members participate in code review, experiment runs, rehearsal, and Q&A preparation. WP1 owns the final report merge; WP4 compiles the slide deck. Each work package is planned at 25% of total effort, and each member presents 10 of the 40 minutes.

7. Risks and mitigation

Risk	Likelihood	Impact	Mitigation
Extreme imbalance makes metrics noisy	High	High	Use PR-AUC and recall at fixed FPR; five seeds; bootstrap confidence intervals; report variance
Cost ratios are assumptions and may drive the conclusion	High	High	Report a cost-ratio sweep (1:1 to 50:1); frame results as sensitivity, not a deployment recommendation
PA-ADMM does not beat cold-start ADMM	Medium	High	Report the component ablation and the negative result; the cost-aware benchmark and regime map remain the contribution
SMO memory at full 284k	High	Medium	Cap SMO at stratified subsets ($n \le 10k$); report the memory wall; use Pegasos/ADMM for full data
Unfair comparison	Medium	High	Same splits, cost ratio, base C, tolerance, hardware, and time budget; report per-iteration and per-time behavior
Kaggle access or data download issues	Medium	Medium	Use the public ULB mirror or OpenML copy; cache the CSV; document the checksum
Schedule slip due to Quiz 2	Medium	High	Front-load core experiments by Saturday; freeze scope; no new algorithms after Saturday

8. Limitations and ethics

The study uses one public, anonymized dataset; the PCA features V1–V28 limit interpretability, and reported fraud labels are incomplete because some fraud is never reported. The cost ratios are illustrative assumptions, not a bank's loss model; the study is a sensitivity analysis and must not be read as financial advice. The method is an empirical hybrid, not a new convergence theory. The study is limited to binary classification, a linear main benchmark, a small RBF subset, a single machine, and stratified subsets for SMO/interior-point. Wall-clock results depend on hardware and will be interpreted with that caveat. No human subjects are involved; code, splits, and seeds will be released, all sources will be cited, and any generative-AI assistance will be acknowledged in accordance with PolyU guidelines.

References

1. Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.

2. Platt, J. C. (1998). Sequential minimal optimization: A fast algorithm for training support vector machines. Microsoft Research Technical Report MSR-TR-98-14.
3. Fan, R.-E., Chen, P.-H., & Lin, C.-J. (2005). Working set selection using second order information for training SVM. *Journal of Machine Learning Research*, 6, 1889–1918.
4. Shalev-Shwartz, S., Singer, Y., Srebro, N., & Cotter, A. (2011). Pegasos: Primal estimated sub-gradient solver for SVM. *Mathematical Programming*, 127(1), 3–30.
5. Boyd, S., Parikh, N., Chu, E., Peleato, B., & Eckstein, J. (2011). Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1), 1–122.
6. Chang, C.-C., & Lin, C.-J. (2011). LIBSVM: A library for support vector machines. *ACM Transactions on Intelligent Systems and Technology*, 2(3), 1–27.
7. Boyd, S., & Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press.
8. Diamond, S., & Boyd, S. (2016). CVXPY: A Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83), 1–5.
9. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
10. Fan, R.-E., Chang, K.-W., Hsieh, C.-J., Wang, X.-R., & Lin, C.-J. (2008). LIBLINEAR: A library for large linear classification. *Journal of Machine Learning Research*, 9, 1871–1874.
11. Hsieh, C.-J., Chang, K.-W., Lin, C.-J., Keerthi, S. S., & Sundararajan, S. (2008). A dual coordinate descent method for large-scale linear SVM. *Proceedings of ICML*, 408–415.
12. Elkan, C. (2001). The foundations of cost-sensitive learning. *Proceedings of IJCAI*, 973–978.
13. Zadrozny, B., Lang, K., & Abe, N. (2003). Cost-sensitive learning by cost-proportionate example weighting. *Proceedings of ICDM*, 435–442.
14. Dal Pozzolo, A., Caelen, O., Le Borgne, Y.-A., Waterschoot, S., & Bontempi, G. (2014). Learned lessons in credit card fraud detection from a practitioner perspective. *Expert Systems with Applications*, 41(10), 4915–4928.
15. Goldstein, T., O'Donoghue, B., Setzer, S., & Baraniuk, R. (2014). Fast alternating direction optimization methods. *SIAM Journal on Imaging Sciences*, 7(3), 1588–1623.

Appendix A. Tentative experiment matrix

The matrix below is the execution plan for the final week; it keeps the comparison fair by fixing splits, cost ratios, and budgets across methods.

Stage	Data / split	Methods	Key settings	Output
Correctness	10k stratified subset	SMO vs LIBSVM; ADMM/PA-ADMM vs CVXPY	$C = 1$, cost ratio 10:1, $\tau = 10^{-4}$	objective gap, accuracy agreement
Main comparison	full 284,807; 5 seeds	Pegasos, ADMM, PA-ADMM; SMO on 10k subset	cost ratios 10/20/50; validated C; same budget	time-to-cost-tolerance, PR-AUC, recall@FPR, cost/example
Component ablation	full data; fixed split	PA-ADMM variants	$K \in \{0,5,10,20\}$; adaptive ρ on/off; $\alpha \in \{1.0,1.5,1.8\}$; ρ_0 sweep	time-to-tolerance, iterations, ρ sensitivity
Imbalance / noise	positive subsampling 0.1–10%; 5/10% label noise	Pegasos, ADMM, PA-ADMM	cost ratio 20:1; 5 seeds	cost/example, PR-AUC, recall@FPR
Temporal robustness	chronological 70/30 split	best two methods	same cost ratio and budget	cost/example, PR-AUC, drift diagnosis
Scaling	synthetic n = 10k–200k	Pegasos, ADMM, PA-ADMM	$d = 20$; cost ratio 10:1	runtime and memory versus n